DATABASES & SQL

THE COMPLETE REFERENCE GUIDE

Database Concepts, Management Systems, SQL & MySQL Workbench Commands Explained

A practical handbook for learning, reference, and daily use

Table of Contents

Chapter 1 Introduction to Databases
Chapter 2 Database Management Systems (DBMS)
Chapter 3 Relational Database Concepts
Chapter 4 Introduction to SQL
Chapter 5 Data Definition Language (DDL) Commands
Chapter 6 Data Manipulation Language (DML) Commands
Chapter 7 Data Query Language (DQL) — SELECT in Depth
Chapter 8 Joins, Keys & Relationships
Chapter 9 Constraints, Indexes & Views
Chapter 10 Transactions & Data Control Language (DCL/TCL)
Chapter 11 Functions, Operators & Data Types
Chapter 12 User & Privilege Management
Chapter 13 MySQL Workbench — Interface & Shortcuts
Chapter 14 Backup, Restore & Maintenance
Chapter 15 Quick-Reference Command Index

Chapter 1

Introduction to Databases

1.1 What is a Database?

A database is an organized collection of structured information, or data, typically stored electronically in a computer system. Databases allow data to be easily accessed, managed, modified, updated, controlled, and organized. Most databases use structured rows and columns for efficient processing and querying of data.

1.2 Why Use a Database Instead of Files?

- **Data integrity:** Rules (constraints) ensure data stays accurate and consistent.
- **Reduced redundancy:** Data is stored once and referenced, avoiding duplication.
- **Concurrent access:** Many users can read/write data at the same time safely.
- **Security:** Access can be restricted per user or role.
- **Scalability:** Databases handle from a few rows to billions of records efficiently.
- **Backup & recovery:** Built-in tools to protect against data loss.

1.3 Types of Databases

Relational Databases (RDBMS)

Data is stored in tables (rows and columns) with relationships defined between them using keys. Examples: MySQL, PostgreSQL, Oracle, Microsoft SQL Server, SQLite.

NoSQL Databases

Designed for unstructured or semi-structured data. Common types include document stores (MongoDB), key-value stores (Redis), column-family stores (Cassandra), and graph databases (Neo4j).

In-Memory Databases

Store data primarily in RAM for extremely fast access, often used for caching (e.g., Redis, Memcached).

1.4 Common Database Terminology

Command	Description
`Table`	A structure that organizes data into rows and columns.
`Row (Record)`	A single entry in a table, representing one item.
`Column (Field)`	A single attribute or property of the data, shared by all rows.

Command	Description
`Primary Key`	A column (or set of columns) that uniquely identifies each row.
`Foreign Key`	A column that references the primary key of another table, creating a relationship.
`Schema`	The structure/blueprint of a database — tables, columns, types, relationships.
`Query`	A request to retrieve or manipulate data, usually written in SQL.
`Index`	A data structure that improves the speed of data retrieval operations.

Chapter 2

Database Management Systems (DBMS)

2.1 What is a DBMS?

A Database Management System (DBMS) is software that enables users to define, create, maintain, and control access to databases. It acts as an interface between the database and the end users or application programs, ensuring data is consistently organized and remains easily accessible.

2.2 Functions of a DBMS

- **Data storage management:** Organizes how data is physically stored on disk.
- **Data manipulation:** Allows insertion, updating, deletion, and retrieval of data.
- **Data security:** Manages user permissions and protects against unauthorized access.
- **Data integrity:** Enforces rules (constraints) so the data remains accurate.
- **Concurrency control:** Ensures multiple users can work simultaneously without conflicts.
- **Backup and recovery:** Provides mechanisms to recover data after failures.
- **Transaction management:** Groups operations so they all succeed or all fail together.

2.3 Types of DBMS

Command	Description
`Hierarchical DBMS`	Organizes data in a tree-like structure with parent-child relationships (e.g., IBM IMS).
`Network DBMS`	Allows more complex relationships with a graph structure where records can have multiple parents.
`Relational DBMS (RDBMS)`	Stores data in tables linked by keys; uses SQL. Most widely used type today (MySQL, PostgreSQL, Oracle).
`Object-Oriented DBMS`	Stores data as objects, similar to object-oriented programming concepts.
`NoSQL DBMS`	Designed for large-scale, distributed data with flexible schemas (MongoDB, Cassandra).

2.4 Popular RDBMS Software

Command	Description
`MySQL`	Open-source RDBMS, widely used for web applications. Managed visually through MySQL Workbench.
`PostgreSQL`	Advanced open-source RDBMS known for standards compliance and extensibility.

Command	Description
`Microsoft SQL Server`	Enterprise RDBMS by Microsoft, managed via SQL Server Management Studio (SSMS).
`Oracle Database`	Enterprise-grade RDBMS widely used in large corporations.
`SQLite`	Lightweight, file-based database engine, often embedded in mobile apps.

2.5 What is MySQL Workbench?

MySQL Workbench is a unified visual tool for database architects, developers, and administrators. It provides data modeling, SQL development, and comprehensive administration tools for server configuration, user administration, backup, and more — all through a graphical interface combined with a SQL editor where commands (like those covered in this guide) are written and executed.

Chapter 3

Relational Database Concepts

3.1 Tables, Rows, and Columns

In a relational database, data is organized into tables. Each table represents one type of entity (e.g., 'customers', 'orders', 'products'). Each row in the table is a single record, and each column represents an attribute of that record, with a defined data type (e.g., text, number, date).

3.2 Keys

Command	Description
`Primary Key (PK)`	Uniquely identifies each row in a table. Cannot contain NULL values and must be unique.
`Foreign Key (FK)`	A column that refers to the primary key in another table, creating a link between the two.
`Composite Key`	A primary key made up of two or more columns combined.
`Candidate Key`	Any column (or set of columns) that could qualify as a primary key.
`Unique Key`	Ensures all values in a column are different, but allows one NULL value (unlike a PK).

3.3 Relationships Between Tables

Command	Description
`One-to-One (1:1)`	Each row in Table A relates to exactly one row in Table B (e.g., a person and their passport).
`One-to-Many (1:N)`	A row in Table A can relate to many rows in Table B (e.g., one customer, many orders).
`Many-to-Many (M:N)`	Rows in Table A can relate to many rows in Table B and vice versa, usually via a junction table (e.g., students and courses).

3.4 Normalization

Normalization is the process of organizing data to reduce redundancy and improve data integrity. It involves dividing large tables into smaller, related tables and defining relationships between them, following a series of rules called 'normal forms'.

Command	Description
`1NF (First Normal Form)`	Each column contains atomic (indivisible) values, and each row is unique.

Command	Description
`2NF (Second Normal Form)`	Meets 1NF and all non-key attributes depend on the whole primary key (no partial dependency).
`3NF (Third Normal Form)`	Meets 2NF and no non-key attribute depends on another non-key attribute (no transitive dependency).
`BCNF`	A stricter version of 3NF that handles certain edge cases of anomalies.

Note: *Normalization reduces duplicate data but can increase the number of joins needed in queries. 'Denormalization' is sometimes used deliberately to improve read performance in reporting systems.*

Chapter 4

Introduction to SQL

4.1 What is SQL?

SQL (Structured Query Language) is the standard language used to communicate with relational databases. It is used to create, read, update, and delete data, as well as define and manage the structure of the database itself.

4.2 Categories of SQL Commands

Command	Description
DDL — Data Definition Language	Defines and modifies database structure: CREATE, ALTER, DROP, TRUNCATE.
DML — Data Manipulation Language	Modifies the data itself: INSERT, UPDATE, DELETE.
DQL — Data Query Language	Retrieves data: SELECT.
DCL — Data Control Language	Controls access and permissions: GRANT, REVOKE.
TCL — Transaction Control Language	Manages transactions: COMMIT, ROLLBACK, SAVEPOINT.

4.3 Connecting to a Database (Workbench)

In MySQL Workbench, a 'connection' stores the host, port, username, and password needed to reach a MySQL server. Once connected, the following basic commands let you explore what is available on the server.

Command	Description
SHOW DATABASES;	Lists all databases (schemas) available on the server.
USE database_name;	Switches the active/default database for subsequent commands.
SELECT DATABASE();	Displays the name of the currently selected database.
SELECT VERSION();	Shows the version of the MySQL server you are connected to.
SELECT CURRENT_USER();	Shows the username currently connected to the server.
SHOW PROCESSLIST;	Displays currently running queries/connections on the server.
SHOW STATUS;	Displays server status variables (e.g., uptime, connections).
SHOW VARIABLES;	Displays system configuration variables for the server.

4.4 Syntax Basics

- SQL statements end with a semicolon `;`
- Keywords (SELECT, FROM, WHERE) are conventionally written in UPPERCASE, though SQL is not case-sensitive for keywords.
- Table and column names should avoid spaces; use underscores instead (e.g., `customer_name`).
- Strings are enclosed in single quotes: `'example text'`
- Comments: `-- this is a comment` or `/* multi-line comment */`

Chapter 5

Data Definition Language (DDL) Commands

5.1 Database-Level Commands

Command	Description
`CREATE DATABASE db_name;`	Creates a new, empty database (schema).
`CREATE DATABASE IF NOT EXISTS db_name;`	Creates the database only if it does not already exist (avoids errors).
`DROP DATABASE db_name;`	Permanently deletes a database and all of its tables and data.
`ALTER DATABASE db_name CHARACTER SET utf8mb4;`	Changes settings of an existing database, such as its character encoding.

5.2 Creating Tables

The CREATE TABLE statement defines a new table, its columns, their data types, and any constraints (such as primary keys).

```
CREATE TABLE customers (
customer_id INT AUTO_INCREMENT PRIMARY KEY,
first_name VARCHAR(50) NOT NULL,
last_name VARCHAR(50) NOT NULL,
email VARCHAR(100) UNIQUE,
created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

5.3 Viewing and Modifying Table Structure

Command	Description
`SHOW TABLES;`	Lists all tables in the currently selected database.
`DESCRIBE table_name;`	Shows the column names, types, and constraints of a table (alias: DESC).
`SHOW CREATE TABLE table_name;`	Displays the full CREATE TABLE statement used to build the table.
`SHOW COLUMNS FROM table_name;`	Lists detailed information about each column in a table.
`RENAME TABLE old_name TO new_name;`	Renames an existing table.

5.4 ALTER TABLE — Modifying Structure

Command	Description
`ALTER TABLE t ADD column_name TYPE;`	Adds a new column to an existing table.
`ALTER TABLE t DROP COLUMN column_name;`	Removes a column and all its data from a table.
`ALTER TABLE t MODIFY column_name NEWTYPE;`	Changes the data type or definition of an existing column.
`ALTER TABLE t CHANGE old_name new_name TYPE;`	Renames a column and/or changes its data type.
`ALTER TABLE t ADD PRIMARY KEY (col);`	Adds a primary key constraint to a column.
`ALTER TABLE t DROP PRIMARY KEY;`	Removes the primary key constraint from a table.
`ALTER TABLE t ADD CONSTRAINT fk FOREIGN KEY (col) REFERENCES other(id);`	Adds a foreign key relationship between two tables.
`ALTER TABLE t RENAME TO new_name;`	Renames the table (alternative to RENAME TABLE).

5.5 Removing Data and Tables

Command	Description
`TRUNCATE TABLE table_name;`	Deletes all rows from a table instantly but keeps the table structure for reuse.
`DROP TABLE table_name;`	Permanently deletes the table itself, including its structure and all data.
`DROP TABLE IF EXISTS table_name;`	Deletes the table only if it exists, preventing an error if it does not.

Note: *DROP and TRUNCATE cannot normally be undone. Always double-check the database you are connected to (SELECT DATABASE();) before running these commands.*

Chapter 6

Data Manipulation Language (DML) Commands

6.1 Inserting Data

Command	Description
`INSERT INTO t (col1, col2) VALUES (v1, v2);`	Adds a new row to a table with specified values for given columns.
`INSERT INTO t VALUES (v1, v2, v3);`	Adds a new row, providing values for every column in table order.
`INSERT INTO t (col1) VALUES (a), (b), (c);`	Inserts multiple rows in a single statement.
`INSERT INTO t2 SELECT * FROM t1;`	Copies rows from one table into another with matching structure.

```
INSERT INTO customers (first_name, last_name, email)
VALUES ('Jane', 'Doe', 'jane.doe@example.com');
```

6.2 Updating Data

Command	Description
`UPDATE t SET col = value WHERE condition;`	Modifies existing rows that match the WHERE condition.
`UPDATE t SET col1 = v1, col2 = v2 WHERE id = 5;`	Updates multiple columns for a specific row.
`UPDATE t1 JOIN t2 ON t1.id = t2.id SET t1.col = t2.col;`	Updates one table's values based on data from a joined table.

```
UPDATE customers
SET email = 'new.email@example.com'
WHERE customer_id = 5;
```

Note: *Always include a WHERE clause with UPDATE unless you intend to change every row in the table. Running UPDATE without WHERE updates the entire table.*

6.3 Deleting Data

Command	Description
`DELETE FROM t WHERE condition;`	Removes rows matching the condition; can be rolled back inside a transaction.
`DELETE FROM t;`	Removes all rows from the table (slower than TRUNCATE, but logged row-by-row).

Command	Description
`DELETE t1 FROM t1 JOIN t2 ON t1.id = t2.id WHERE condition;`	Deletes rows from one table based on a condition involving a joined table.

```
DELETE FROM customers
WHERE customer_id = 12;
```

6.4 REPLACE and UPSERT-style Statements

Command	Description
`REPLACE INTO t (...) VALUES (...);`	Deletes the old row (if a duplicate key exists) and inserts the new one.
`INSERT ... ON DUPLICATE KEY UPDATE col=value;`	Inserts a new row, or updates the existing row if a unique/primary key conflict occurs.

```
INSERT INTO products (id, name, stock)
VALUES (1, 'Widget', 50)
ON DUPLICATE KEY UPDATE stock = stock + 50;
```

Chapter 7

Data Query Language (DQL) — SELECT in Depth

7.1 Basic SELECT Statements

Command	Description
`SELECT * FROM t;`	Retrieves all columns and all rows from a table.
`SELECT col1, col2 FROM t;`	Retrieves only the specified columns.
`SELECT DISTINCT col FROM t;`	Retrieves unique values only, removing duplicates.
`SELECT col AS alias FROM t;`	Renames a column in the result using an alias.
`SELECT col1, col2 FROM t WHERE condition;`	Filters rows based on a condition before returning results.

7.2 Filtering with WHERE

Command	Description
`WHERE col = value`	Matches rows where the column equals a specific value.
`WHERE col != value / <> value`	Matches rows where the column does NOT equal a value.
`WHERE col > value / < value`	Matches rows with values greater than or less than a number/date.
`WHERE col BETWEEN a AND b`	Matches values within an inclusive range.
`WHERE col IN (a, b, c)`	Matches rows where the column equals any value in the list.
`WHERE col LIKE '%abc%'`	Pattern matching: % means any sequence of characters, _ means a single character.
`WHERE col IS NULL / IS NOT NULL`	Checks whether a column has no value (NULL) or has a value.
`WHERE cond1 AND cond2`	Both conditions must be true.
`WHERE cond1 OR cond2`	At least one condition must be true.
`WHERE NOT condition`	Negates a condition.

7.3 Sorting and Limiting Results

Command	Description
`ORDER BY col ASC;`	Sorts results in ascending order (default).

Command	Description
`ORDER BY col DESC;`	Sorts results in descending order.
`ORDER BY col1, col2 DESC;`	Sorts by multiple columns; later columns break ties from earlier ones.
`LIMIT n;`	Restricts the result set to the first n rows.
`LIMIT n OFFSET m;`	Skips the first m rows, then returns the next n rows (used for pagination).

7.4 Grouping and Aggregation

Command	Description
`GROUP BY col;`	Groups rows that share the same value in a column, for use with aggregate functions.
`HAVING condition;`	Filters groups after aggregation (used instead of WHERE for aggregated values).
`COUNT(col)`	Returns the number of rows (or non-NULL values in a column).
`SUM(col)`	Returns the total sum of a numeric column.
`AVG(col)`	Returns the average value of a numeric column.
`MIN(col) / MAX(col)`	Returns the smallest / largest value in a column.

```
SELECT department, COUNT(*) AS employee_count, AVG(salary) AS avg_salary
FROM employees
GROUP BY department
HAVING COUNT(*) > 5
ORDER BY avg_salary DESC;
```

7.5 Subqueries

A subquery is a SELECT statement nested inside another query. It can appear in the WHERE, FROM, or SELECT clause and is often used to filter results based on the outcome of another query.

```
SELECT name, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

Chapter 8

Joins, Keys & Relationships

8.1 Why Use Joins?

Joins combine rows from two or more tables based on a related column between them, allowing you to retrieve connected data — for example, an order along with the name of the customer who placed it.

8.2 Types of Joins

Command	Description
`INNER JOIN`	Returns only rows that have matching values in both tables.
`LEFT JOIN (LEFT OUTER JOIN)`	Returns all rows from the left table, plus matched rows from the right table; unmatched right-side columns are NULL.
`RIGHT JOIN (RIGHT OUTER JOIN)`	Returns all rows from the right table, plus matched rows from the left table; unmatched left-side columns are NULL.
`FULL OUTER JOIN`	Returns all rows from both tables, with NULLs where there is no match (not directly supported in MySQL; emulated with UNION).
`CROSS JOIN`	Returns the Cartesian product — every row from the first table combined with every row from the second.
`SELF JOIN`	Joins a table to itself, often used for hierarchical or comparison data.

```
-- INNER JOIN example
SELECT orders.order_id, customers.first_name, customers.last_name
FROM orders
INNER JOIN customers ON orders.customer_id = customers.customer_id;

-- LEFT JOIN example
SELECT customers.first_name, orders.order_id
FROM customers
LEFT JOIN orders ON customers.customer_id = orders.customer_id;
```

8.3 UNION and UNION ALL

Command	Description
`SELECT ... UNION SELECT ...;`	Combines results of two queries, removing duplicate rows.
`SELECT ... UNION ALL SELECT ...;`	Combines results of two queries, keeping all duplicates (faster than UNION).

8.4 Foreign Keys in Practice

```
CREATE TABLE orders (
order_id INT AUTO_INCREMENT PRIMARY KEY,
customer_id INT,
order_date DATE,
CONSTRAINT fk_customer
FOREIGN KEY (customer_id)
REFERENCES customers(customer_id)
ON DELETE CASCADE
ON UPDATE CASCADE
);
```

Command	Description
`ON DELETE CASCADE`	If the referenced row in the parent table is deleted, related rows here are deleted too.
`ON DELETE SET NULL`	If the referenced row is deleted, the foreign key column here is set to NULL.
`ON UPDATE CASCADE`	If the referenced key value changes, this column updates automatically to match.
`ON DELETE RESTRICT`	Prevents deletion of the referenced row if related rows exist here (default behavior).

Chapter 9

Constraints, Indexes & Views

9.1 Column Constraints

Command	Description
`NOT NULL`	Ensures a column cannot store a NULL (empty) value.
`UNIQUE`	Ensures all values in a column are different from one another.
`PRIMARY KEY`	Combination of NOT NULL and UNIQUE; uniquely identifies each row.
`FOREIGN KEY`	Links a column to the primary key of another table for referential integrity.
`CHECK (condition)`	Ensures all values in a column satisfy a specific condition.
`DEFAULT value`	Sets a default value for a column when none is specified during INSERT.
`AUTO_INCREMENT`	Automatically generates a unique number for each new row (commonly used with primary keys).

9.2 Working with Indexes

An index is a separate data structure that the database engine maintains to speed up data retrieval. Indexes act like a 'table of contents' for a table, allowing the engine to find rows without scanning every record. The trade-off is that indexes use extra storage and slightly slow down INSERT/UPDATE/DELETE operations because the index must also be updated.

Command	Description
`CREATE INDEX idx_name ON t (col);`	Creates a standard index on a column to speed up searches and joins.
`CREATE UNIQUE INDEX idx_name ON t (col);`	Creates an index that also enforces uniqueness of values in the column.
`SHOW INDEX FROM t;`	Lists all indexes defined on a table.
`DROP INDEX idx_name ON t;`	Removes an index from a table.
`ALTER TABLE t ADD INDEX idx_name (col);`	Alternative syntax to add an index via ALTER TABLE.

9.3 Views

A view is a saved SQL query that acts like a virtual table. Views can simplify complex queries, restrict access to specific columns/rows, and present data in a consistent format without duplicating it.

Command	Description
`CREATE VIEW view_name AS SELECT ...;`	Creates a virtual table based on a stored SELECT query.
`SELECT * FROM view_name;`	Queries a view exactly like a regular table.
`CREATE OR REPLACE VIEW view_name AS SELECT ...;`	Updates an existing view's definition, or creates it if it doesn't exist.
`DROP VIEW view_name;`	Deletes a view (does not affect the underlying tables/data).
`SHOW FULL TABLES WHERE table_type = 'VIEW';`	Lists all views in the current database.

```
CREATE VIEW active_customers AS
SELECT customer_id, first_name, last_name, email
FROM customers
WHERE status = 'active';
```

Chapter 10

Transactions & Data Control Language (DCL/TCL)

10.1 What is a Transaction?

A transaction is a sequence of one or more SQL operations executed as a single unit of work. Transactions follow the ACID properties to guarantee reliability even if errors, crashes, or concurrent access occur.

Command	Description
Atomicity	All operations in the transaction succeed, or none of them are applied.
Consistency	The database moves from one valid state to another, never leaving data in an invalid state.
Isolation	Concurrent transactions do not interfere with each other's intermediate results.
Durability	Once a transaction is committed, the changes are permanent, even after a crash.

10.2 Transaction Control Commands (TCL)

Command	Description
START TRANSACTION; / BEGIN;	Marks the beginning of a new transaction.
COMMIT;	Saves all changes made during the current transaction permanently.
ROLLBACK;	Undoes all changes made during the current transaction since the last COMMIT.
SAVEPOINT savepoint_name;	Creates a named point within a transaction that you can roll back to.
ROLLBACK TO savepoint_name;	Reverts changes back to a specific savepoint without ending the whole transaction.
SET autocommit = 0;	Disables automatic committing of each statement, requiring explicit COMMIT.
SET autocommit = 1;	Re-enables automatic committing after each statement (MySQL default).

```
START TRANSACTION;

UPDATE accounts SET balance = balance - 100 WHERE id = 1;
UPDATE accounts SET balance = balance + 100 WHERE id = 2;

COMMIT; -- or ROLLBACK; if something went wrong
```

10.3 Data Control Language (DCL)

DCL commands manage permissions and access rights, determining what each user is allowed to do within the database.

Command	Description
`GRANT privilege ON object TO user;`	Gives a user specific permissions on a database, table, or other object.
`REVOKE privilege ON object FROM user;`	Removes previously granted permissions from a user.
`SHOW GRANTS FOR user;`	Displays the privileges currently assigned to a user.

Chapter 11

Functions, Operators & Data Types

11.1 Common MySQL Data Types

Command	Description
`INT / BIGINT`	Whole numbers; BIGINT for very large numbers.
`DECIMAL(p,s) / FLOAT / DOUBLE`	Numbers with decimals; DECIMAL is exact (good for money), FLOAT/DOUBLE are approximate.
`VARCHAR(n)`	Variable-length text up to n characters.
`CHAR(n)`	Fixed-length text, padded with spaces to length n.
`TEXT`	Large variable-length text, for long content like articles or descriptions.
`DATE`	Stores a date value (YYYY-MM-DD).
`DATETIME / TIMESTAMP`	Stores date and time; TIMESTAMP is often used for auto-updating record times.
`BOOLEAN / TINYINT(1)`	Stores true/false values (MySQL stores these as 0 or 1).
`ENUM('a','b','c')`	A string column restricted to one of a predefined list of values.
`JSON`	Stores JSON-formatted data, allowing flexible/semi-structured fields.

11.2 String Functions

Command	Description
`CONCAT(a, b, ...)`	Joins two or more strings together into one.
`UPPER(str) / LOWER(str)`	Converts text to uppercase or lowercase.
`LENGTH(str)`	Returns the number of characters (bytes) in a string.
`TRIM(str)`	Removes leading and trailing spaces from a string.
`SUBSTRING(str, start, length)`	Extracts a portion of a string starting at a given position.
`REPLACE(str, from, to)`	Replaces all occurrences of one substring with another.

11.3 Numeric Functions

Command	Description
`ROUND(num, decimals)`	Rounds a number to a specified number of decimal places.
`CEIL(num) / FLOOR(num)`	Rounds a number up or down to the nearest whole number.

Command	Description
`ABS(num)`	Returns the absolute (non-negative) value of a number.
`MOD(a, b)`	Returns the remainder of dividing a by b.
`POWER(a, b)`	Raises a to the power of b.

11.4 Date & Time Functions

Command	Description
`NOW()`	Returns the current date and time.
`CURDATE() / CURTIME()`	Returns the current date, or the current time, separately.
`DATE_ADD(date, INTERVAL n unit)`	Adds a time interval (days, months, etc.) to a date.
`DATEDIFF(date1, date2)`	Returns the number of days between two dates.
`YEAR(date) / MONTH(date) / DAY(date)`	Extracts the year, month, or day from a date value.
`DATE_FORMAT(date, format)`	Formats a date value as a string according to a given pattern.

11.5 Conditional Logic Functions

Command	Description
`IF(condition, value_if_true, value_if_false)`	Returns one of two values depending on whether a condition is true.
`IFNULL(expr, default)`	Returns a default value if the expression is NULL.
`COALESCE(val1, val2, ...)`	Returns the first non-NULL value from a list.
`CASE WHEN ... THEN ... ELSE ... END`	Performs multi-branch conditional logic, similar to if/else if/else.

```
SELECT name,
CASE
WHEN salary >= 5000 THEN 'High'
WHEN salary >= 2000 THEN 'Medium'
ELSE 'Low'
END AS salary_band
FROM employees;
```

Chapter 12

User & Privilege Management

12.1 Creating and Managing Users

Command	Description
`CREATE USER 'user'@'host' IDENTIFIED BY 'password';`	Creates a new database user account with a password, restricted to connect from 'host'.
`ALTER USER 'user'@'host' IDENTIFIED BY 'newpassword';`	Changes the password for an existing user.
`RENAME USER 'old'@'host' TO 'new'@'host';`	Renames a user account.
`DROP USER 'user'@'host';`	Permanently deletes a user account.
`SELECT user, host FROM mysql.user;`	Lists all user accounts on the server and the hosts they can connect from.

12.2 Granting and Revoking Privileges

Command	Description
`GRANT ALL PRIVILEGES ON db.* TO 'user'@'host';`	Gives a user full control over every table in a specific database.
`GRANT SELECT, INSERT ON db.table TO 'user'@'host';`	Gives a user permission to read and insert data into a specific table only.
`GRANT ALL PRIVILEGES ON *.* TO 'user'@'host';`	Gives a user administrative privileges over the entire server (use with caution).
`REVOKE INSERT ON db.table FROM 'user'@'host';`	Removes a specific privilege from a user.
`FLUSH PRIVILEGES;`	Reloads the privilege tables so that GRANT/REVOKE changes take effect immediately.
`SHOW GRANTS FOR 'user'@'host';`	Displays all privileges currently granted to a user.

12.3 The host part: 'user'@'host'

Command	Description
`'user'@'localhost'`	User can only connect from the same machine the server is running on.
`'user'@'%'`	User can connect from any host/IP address (use carefully — security risk if not paired with a strong password).

Command	Description
`'user'@'192.168.1.10'`	User can only connect from a specific IP address.

> ***Note:*** *Always run FLUSH PRIVILEGES; after manually editing the mysql.user table or running GRANT/REVOKE statements outside of normal account-management commands, to ensure the server recognizes the changes.*

Chapter 13

MySQL Workbench — Interface & Shortcuts

13.1 Main Areas of the Interface

Command	Description
`SQL Editor`	The main area where you type and run SQL statements/scripts.
`Navigator Panel`	Left-hand sidebar showing schemas, tables, views, and stored routines.
`Output / Action Output Panel`	Bottom panel showing the results, errors, and status of executed statements.
`Result Grid`	Displays query results in a spreadsheet-like grid, which can be edited directly.
`Object Browser`	Lets you browse and manage tables, columns, indexes, and triggers visually.
`ER Diagram (EER Model)`	A visual tool for designing database schemas and relationships graphically.

13.2 Keyboard Shortcuts

Command	Description
`Ctrl + Enter (Cmd + Return on Mac)`	Executes the current SQL statement (the one where the cursor is placed).
`Ctrl + Shift + Enter`	Executes the entire SQL script in the editor.
`Ctrl + /`	Comments or uncomments the selected line(s) of SQL.
`Ctrl + S`	Saves the current SQL script/file.
`Ctrl + T`	Opens a new SQL tab/query editor.
`Ctrl + Space`	Triggers SQL autocomplete suggestions.
`F5`	Refreshes the schema list in the Navigator panel.
`Ctrl + F`	Opens the find/search bar within the SQL editor.
`Ctrl + Z / Ctrl + Y`	Undo / redo the last edit in the SQL editor.

13.3 Visual Database Design

- **Database → Reverse Engineer:** Generates an EER diagram from an existing database.
- **Database → Forward Engineer:** Generates SQL CREATE scripts from an EER diagram.
- **Database → Synchronize Model:** Compares and syncs a model with a live database.

- **File → Export → Forward Engineer SQL CREATE Script:** Saves the model as a .sql file.

13.4 Server Administration Tab

- **Server Status:** View uptime, connections, and performance metrics.
- **Client Connections:** View and kill active connections/sessions.
- **Users and Privileges:** Visual interface for managing users (equivalent to CREATE USER / GRANT).
- **Export / Import:** Visual interface for backing up and restoring databases (uses mysqldump under the hood).
- **Options File:** Edit server configuration variables (my.cnf / my.ini) visually.

Chapter 14

Backup, Restore & Maintenance

14.1 Backing Up with mysqldump (Command Line)

Command	Description
`mysqldump -u user -p db_name > backup.sql`	Exports an entire database's structure and data to a .sql file.
`mysqldump -u user -p db_name table_name > table.sql`	Exports a single table's structure and data.
`mysqldump -u user -p --no-data db_name > structure.sql`	Exports only the table structures, without any data.
`mysqldump -u user -p --all-databases > all_db.sql`	Exports every database on the server into one file.

14.2 Restoring from a Backup

Command	Description
`mysql -u user -p db_name < backup.sql`	Restores a database from a .sql backup file (database must already exist).
`SOURCE /path/to/backup.sql;`	Runs a .sql script file from within the MySQL command-line client or Workbench.

14.3 Workbench Export/Import Wizard

MySQL Workbench provides a graphical Data Export and Data Import/Restore tool under the Server Administration tab. It allows selecting specific schemas or tables, choosing between SQL dump formats, and scheduling exports — all without typing command-line commands.

14.4 Table Maintenance Commands

Command	Description
`ANALYZE TABLE table_name;`	Updates index statistics to help the optimizer choose efficient query plans.
`OPTIMIZE TABLE table_name;`	Reclaims unused space and defragments table data, improving performance.
`CHECK TABLE table_name;`	Checks a table for errors or corruption.
`REPAIR TABLE table_name;`	Attempts to fix a corrupted table (mainly for MyISAM tables).

Command	Description
`SHOW TABLE STATUS;`	Displays information about all tables, including size, row count, and engine.

14.5 Storage Engines

Command	Description
`InnoDB`	Default MySQL engine; supports transactions, foreign keys, and row-level locking. Recommended for most use cases.
`MyISAM`	Older engine; faster for read-heavy workloads but lacks transaction support and foreign keys.
`MEMORY`	Stores table data in RAM for very fast access; data is lost when the server restarts.

```
CREATE TABLE logs (
id INT AUTO_INCREMENT PRIMARY KEY,
message VARCHAR(255)
) ENGINE = InnoDB;
```

Chapter 15

Quick-Reference Command Index

15.1 Most Frequently Used Commands

Command	Description
`SHOW DATABASES;`	List all databases on the server.
`USE db_name;`	Select a database to work with.
`SHOW TABLES;`	List all tables in the selected database.
`DESCRIBE table_name;`	Show a table's columns and types.
`SELECT * FROM table_name;`	Retrieve all data from a table.
`SELECT col FROM table_name WHERE condition;`	Retrieve filtered data.
`INSERT INTO table_name (...) VALUES (...);`	Add a new row of data.
`UPDATE table_name SET col=val WHERE condition;`	Modify existing data.
`DELETE FROM table_name WHERE condition;`	Remove specific data.
`CREATE TABLE table_name (...);`	Create a new table.
`ALTER TABLE table_name ADD col TYPE;`	Add a column to a table.
`DROP TABLE table_name;`	Delete a table permanently.
`CREATE INDEX idx ON table_name (col);`	Speed up searches on a column.
`JOIN ... ON ...`	Combine data from multiple tables.
`GROUP BY / HAVING`	Aggregate and filter grouped data.
`ORDER BY / LIMIT`	Sort and restrict result sets.
`START TRANSACTION / COMMIT / ROLLBACK`	Control grouped operations safely.
`GRANT / REVOKE`	Manage user permissions.
`CREATE USER / DROP USER`	Manage database accounts.

15.2 General SQL Writing Tips

- Always test SELECT statements with the same WHERE clause before running UPDATE or DELETE.
- Use transactions (START TRANSACTION / COMMIT / ROLLBACK) when making multiple related changes.
- Name tables in plural (customers, orders) and primary keys consistently (id or table_id).
- Add indexes to columns frequently used in WHERE, JOIN, and ORDER BY clauses.
- Back up your database before performing structural changes (ALTER, DROP).
- Use comments (-- or /* */) to document complex queries for future reference.

End of guide. This reference covers the core concepts and commands needed for everyday database design, querying, and administration using SQL and MySQL Workbench.